



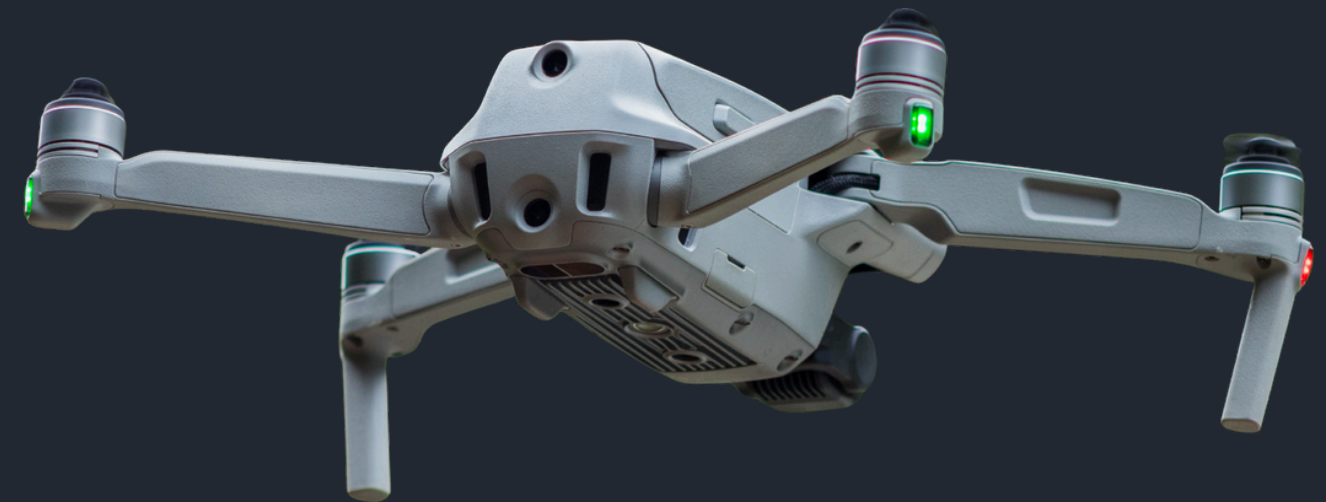
AeroPath

Drone Delivery System

Rodaina Sabry

Mohamed Magdy

Basel Atef



💡 Idea

This project is a software system that simulates drone delivery operations using Python.

It focuses on managing drones, packages, and delivery missions in a simple environment.

💡 Problem

Drone delivery is not a simple task.

It involves multiple challenges such as limited battery capacity, different package weights,
and the need to determine an efficient path to the destination.



Challenges

The system must ensure that:



- 01.** The drone has enough battery for the full trip
- 02.** A valid path exists between points
- 03.** The delivery is completed safely without failure

🔧 Solution

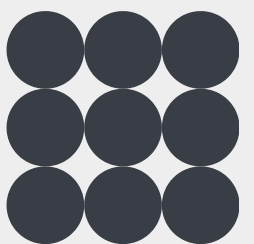
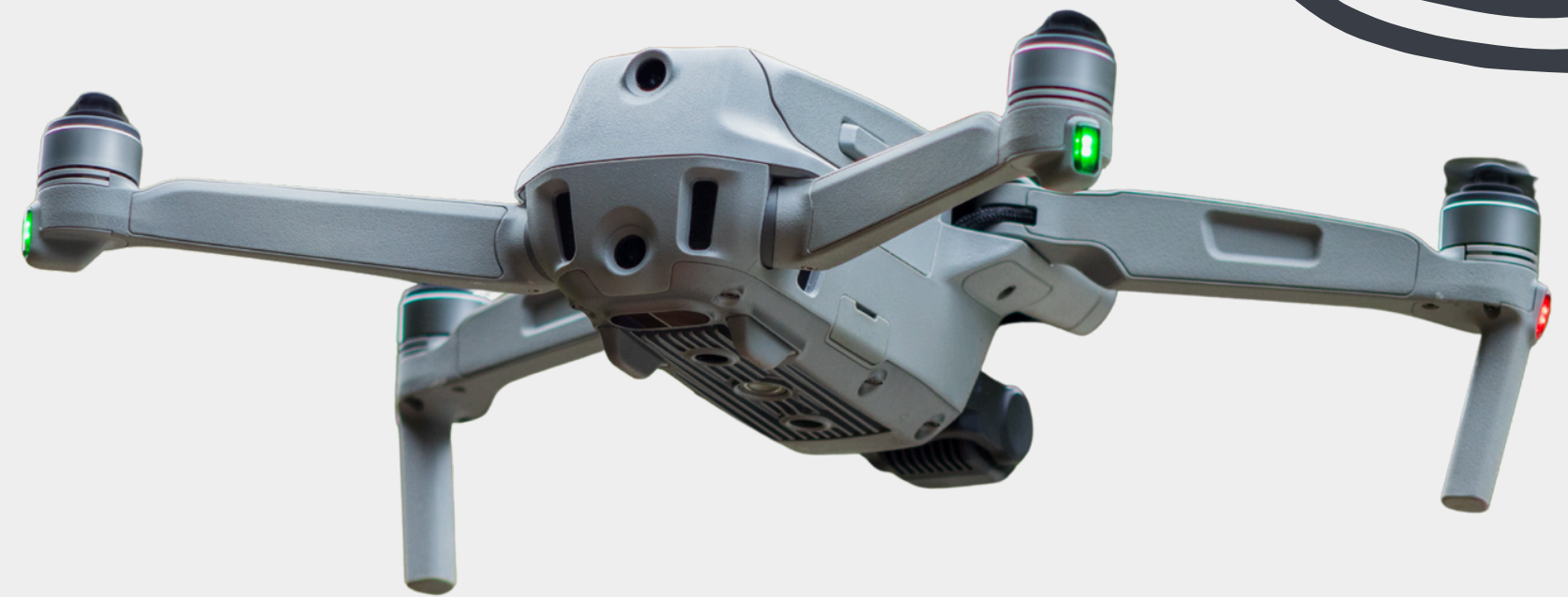
To solve this problem, we designed a system that:

- Represents drones and packages using classes
- Assigns each package a fixed destination
- Simulates the drone movement step-by-step
- Calculates battery usage during the trip

🔧 System Components

The system is built using four main components:

- Drone: represents the flying unit
- Package: contains delivery information
- Fleet: manages all drones and packages
- Mission: controls the delivery process



🔌 classes

🔌 Drone

```
from enum import Enum

class DroneStatus(Enum):
    IDLE = "idle"
    MOVING = "moving"
    DELIVERING = "delivering"

class Drone:
    def __init__(self, drone_id, max_weight):
        self.id = drone_id
        self.max_weight = max_weight
        self.battery = 100
        self.position = (0, 0)
        self.status = DroneStatus.IDLE

    def set_status(self, status):
        if isinstance(status, DroneStatus):
            self.status = status

    def consume_battery(self, amount):
        self.battery = max(0, self.battery - amount)

    def __str__(self):
        return f"{self.id} | {self.status.value} | {self.battery}"
```

🔌 Package

```
class Package:
    def __init__(self, package_id, weight, destination):
        self.id = package_id
        self.weight = weight
        self.destination = destination

    def __str__(self):
        return f"{self.id} | {self.weight} | {self.destination}"
```


class fleet

```
class Fleet:
    def save_data(self, filename="fleet.json"):
        with open(filename, "w") as file:
            json.dump(data, file, indent=4)

    def load_data(self, filename="fleet.json"):
        try:
            with open(filename, "r") as file:
                data = json.load(file)

            for d in data["drones"]:
                drone = Drone(d["id"], d["max_weight"])
                drone.battery = d["battery"]
                drone.position = tuple(d["position"])
                drone.status = DroneStatus(d["status"])
                self.drones.append(drone)

            for p in data["packages"]:
                package = Package(
                    p["id"],
                    p["weight"],
                    tuple(p["destination"])
                )
                self.packages.append(package)

        except FileNotFoundError:
            print("No saved data found")
```

```
class Fleet:
    def __init__(self):
        self.drones = []
        self.packages = []

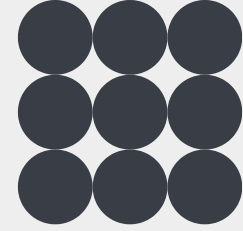
    def add_drone(self, drone):
        self.drones.append(drone)

    def add_package(self, package):
        self.packages.append(package)

    def save_data(self, filename="fleet.json"):
        data = {
            "drones": [
                {
                    "id": d.id,
                    "max_weight": d.max_weight,
                    "battery": d.battery,
                    "position": d.position,
                    "status": d.status.value
                }
                for d in self.drones
            ],
            "packages": [
                {
                    "id": p.id,
                    "weight": p.weight,
                    "destination": p.destination
                }
                for p in self.packages
            ]
        }

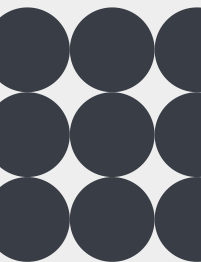
        with open(filename, "w") as file:
            json.dump(data, file, indent=4)
```

⚡ Mission



```
1 from pathfinding import find_path
2 from models.drone import DroneStatus
3
4
5 def battery_use(steps, weight):
6     return steps * (1 + weight * 0.1)
7
8
9 def can_fly(drone, required_battery):
10     return drone.battery >= required_battery
11
12 def start_trip(drone, package, grid):
13
14     path = find_path(drone.position, package.destination, grid)
15
16     if not path:
17         print("No path found")
18         return False
19
20     steps = len(path)
21
22     battery_needed = battery_use(steps, package.weight)
23
24     total_battery = battery_needed * 2
25
26     if not can_fly(drone, total_battery):
27         print("Battery not enough")
28         return False
29
30     drone.set_status(DroneStatus.MOVING)
31
32
33     for step in path:
34         drone.position = step
35         drone.consume_battery(1 + package.weight * 0.1)
```

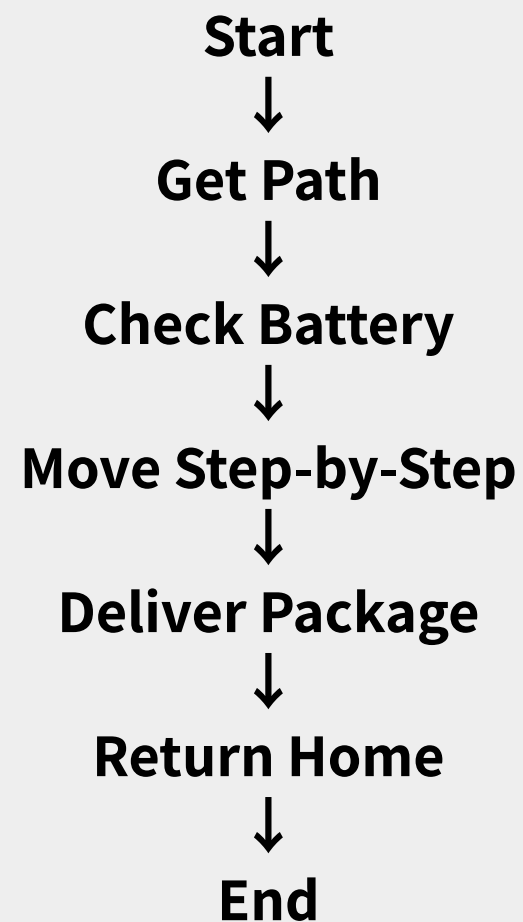
```
        print(f"Drone moving to {step} | Battery: {drone.battery:.1f}%")
        if drone.battery <= 10:
            print("Low battery! Going home...")
            go_home(drone, grid)
            return False
        drone.set_status(DroneStatus.DELIVERING)
        print("Package delivered")
        go_home(drone, grid)
        return True
def go_home(drone, grid):
    home_path = find_path(drone.position, (0, 0), grid)
    if not home_path:
        print("Can't return home")
        return
    for step in home_path:
        drone.position = step
        drone.consume_battery(1)
        print(f"Returning home: {step} | Battery: {drone.battery:.1f}%")
    drone.set_status(DroneStatus.IDLE)
    print("Drone returned home safely")
```



Algorithm

The delivery process follows a clear sequence:

- The system finds a path from the drone to the destination
- It calculates the required battery
- If sufficient, the drone starts moving step-by-step
- After delivery, the drone returns to its starting point



path finding

```
45
46 def find_path(start, goal):
47     my_queue = []
48     came_from = {}
49     path = []
50     steps = 0
51     total_distance = 0
52     visited = set()
53
54     heapq.heappush(my_queue, (0, start))
55
56     while my_queue:
57         f, current = heapq.heappop(my_queue)
58
59         if current == goal:
60             break
61
62         if current in visited:
63             continue
64
65         visited.add(current)
66
67         neighbors = get_neighbors(current, grid_size, obstacles)
68         steps += 1
69
70         for i in neighbors:
71
72             if i not in visited:
73                 remaining_distance = node_distance(i, goal)
74                 total_distance = steps + remaining_distance
75                 heapq.heappush(my_queue, (total_distance, i))
76
77                 if i not in came_from:
78                     came_from[i] = current
79
80
```

```
7 def add_horizontal_obstacle(x1, x2, y):
8     return [(x, y) for x in range(x1, x2 + 1)]
9
10 def add_vertical_obstacle(x, y1, y2):
11     return [(x, y) for y in range(y1, y2 + 1)]
12
13 def add_rectangle_obstacle(x1, x2, y1, y2):
14     return [(x, y) for x in range(x1, x2 + 1) for y in range(y1, y2 + 1)]
15
16 obstacles.update(add_horizontal_obstacle(2, 5, 3))
17 obstacles.update(add_rectangle_obstacle(4,6,3,4))
18 obstacles.update(add_vertical_obstacle(1,2,5))
19 obstacles.update(add_horizontal_obstacle(1,4,6))
20
21
22
23 def get_neighbors(current, grid_size, obstacles):
24     neighbors = []
25     x = current[0]
26     y = current[1]
27     width = grid_size[0]
28     height = grid_size[1]
29
30     if x+1 < width and (x+1,y) not in obstacles:
31         neighbors.append((x+1,y))
32     if x-1 >= -width and (x-1,y) not in obstacles:
33         neighbors.append((x-1,y))
34     if y+1 < height and (x,y+1) not in obstacles:
35         neighbors.append((x,y+1))
36     if y-1 >= -height and (x,y-1) not in obstacles:
37         neighbors.append((x,y-1))
38
39     return neighbors
40
41
42 def node_distance(node, goal):
43     return abs(goal[0]-node[0]) + abs(goal[1]-node[1])
44
```



↳ path finding algorithm



A*

Find shortest path between two point by choosing lowest cost point

Distance = Steps + Remaining (Manhattan Distance)



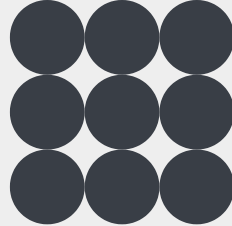
Manhattan distance = $|X_{\text{goal}} - X_{\text{node}}| + |Y_{\text{goal}} - Y_{\text{node}}|$

Periority queue = [(distance , (x,y)).....]

Filling “came_from” dictionary

Filling path & reversing





Visualization: Matplotlib



We use matplotlib to initialize a graph-style graphic and plot points corresponding to the path and drone

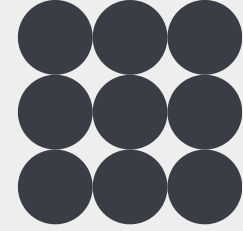
We draw obstacles by representing them on a 2D Matrix and highlighting their cells in the graph

By getting the path, which is an array of points, we can use it to plot these points as lines

We finally use interpolation between each point to generate smooth frames for the drone point to loop through, making a smooth animation.



Visualization



```
29 def drawPath(ax, path, placeMarker=True)
30     xPoints = [x for [x, y] in path]
31     yPoints = [y for [x, y] in path]
32
33     (goalX, goalY) = path[-1]
34     ax.plot(xPoints, yPoints, color="tabularblue")
35     if placeMarker:
36         ax.plot([goalX], [goalY], "g^",
37
38
39 def drawObstacleGrid(ax, cmap, obstacles)
40     grid = np.zeros((100, 100))
41
42     for (x, y) in obstacles:
43         grid[49-y][x+50] = 1
44
45     ax.imshow(grid, cmap=cmap, extent=[
46
```

```
113 def simulateDrone(drone, package, path):
114     fig, ax = plt.subplots(layout="constrained", figsize=(8, 8))
115     manager = plt.get_current_fig_manager()
116
117     limitNum = 20
118     limitRange = [x for x in range(-limitNum, limitNum+1, 2)]
119
120     cmap = ListedColormap([(0, 0, 0, 0), (0, 0, 0, 1)])
121
122     ax.grid()
123     ax.set_aspect("equal")
124     ax.set_xlim(-limitNum, limitNum)
125     ax.set_ylim(-limitNum, limitNum)
126     ax.set_xticks(limitRange)
127     ax.set_yticks(limitRange)
128
129     manager.window.geometry("800x800+900+120")
130
131     ax.scatter([0], [0], marker="x", c="#FFD700", s=100, zorder=9)
132
133     drawObstacleGrid(ax, cmap, pathFinding.obstacles)
134     uiDrone = UIDrone(ax, drone.drone_id, drone.max_weight)
135     uiDrone.battery = drone.battery
136     uiPath = path
137     anim = executePathwalk(fig, ax, uiDrone, package, uiPath, TIME_RUNNING, PAUSE_TIME)
138     plt.show()
139
140
```

```
36 def drawObstacleGrid(ax, cmap, obstacles):
37     grid = np.zeros((100, 100))
38
39     for (x, y) in obstacles:
40         grid[49-y][x+50] = 1
41
42     ax.imshow(grid, cmap=cmap, extent=[-50.5, 49.5, -50.5, 49.5])
43
44
```